
ROADASSIST BHARAT

D3 - AI & Data Science Lead

Individual Roadmap

| *One Platform. Every Vehicle. Every Phone. Every Road.*

Document docs/04-ai-lead-roadmap.md

Version 1.0 — planning baseline

Team 4 engineers · 22 weeks · 11 sprints · 18 phases

Status Baseline (frozen at Sprint 0 exit)

D3 — AI & Data Science Lead · Individual Roadmap

Owens: all 9 AI subsystems, model training and serving, on-device inference, feature store, voice/IVR NLU, analytics pipeline, government forecasting. **Backs up:** D1 on the analytics service. **Total allocation:** 22 weeks · ~176 ideal days.

Standing responsibilities: - **Define the baseline before you train.** A model that has no documented baseline cannot be said to work. - Every model ships with a **model card** and a **fallback**. No exceptions, including for models you're proud of. - No model is ever on the critical path for correctness or safety. Models advise; rules decide. - Latency and size budgets are hard constraints, not aspirations. A 94%-accurate model that takes 4 seconds on a 2 GB phone is a failed model.

The Nine AI Systems — Specification Summary

#	System	Inputs	Outputs	Approach	Where it runs	Baseline to beat
1	Vehicle Diagnosis AI	Symptom text/voice, vehicle model, DTC codes, telemetry, history	Ranked probable causes + severity + drivability + parts	Retrieval (BM25+dense) over a curated fault KB → gradient-boosted classifier → LLM for the natural-language explanation only	Server (LLM), on-device (classifier)	Rules engine on DTC lookup: top-3 acc. 0.62
2	Image Diagnosis AI	1–5 photos	Damage type, affected part, severity, repair estimate	Fine-tuned vision backbone (EfficientNetV2-S / MobileViT), multi-label + severity head	Server; quantized on-device variant	Human non-expert labelling: top-3 acc. 0.55
3	Voice Assistant	Speech, 8 languages + code-switching	Intent + slots + spoken response	Whisper-small fine-tuned on Indian-accented speech → intent classifier + slot filler → TTS	Server; on-device keyword spotting for "SOS"	Keyword matching: intent acc. 0.48
4	Predictive Maintenance	Telemetry time series, service history, model, usage pattern	Failure probability per component over 30/90 days + recommended service date	Survival analysis (Cox / DeepSurv) + LightGBM per component	Server, batch nightly	Fixed manufacturer service interval: recall@30d 0.31
5	Smart Mechanic Matching	Request context, mechanic features, geo, historical outcomes	Ranked mechanic list with acceptance probability + predicted ETA	Learning-to-rank (LambdaMART) + separate ETA regressor	Server, <300 ms	Distance + rating sort: acceptance@1 0.41
6	Crash Detection	Accelerometer, gyroscope, GPS speed, barometer (100 Hz windows)	Crash probability + severity estimate	1D CNN + GRU on sensor windows, INT8 quantized	On-device only	Threshold on peak G-force: FPR 1 per 12 driving hours

#	System	Inputs	Outputs	Approach	Where it runs	Baseline to beat
7	Fraud Detection	Booking graph, payment patterns, device fingerprints, GPS traces, review patterns	Risk score + reason codes	Graph features + isolation forest + supervised GBM on confirmed cases	Server, async	Manual review rules: precision 0.34
8	Government Analytics	Historical incidents, weather, traffic, road geometry, events	Spatiotemporal risk forecast, blackspot ranking, demand prediction	ST-GNN / temporal fusion transformer + spatial clustering	Server, batch	Historical-average-by-location: MAE baseline recorded in P13
9	Offline AI Bundle	Symptoms, DTC, sensors	Diagnosis + crash detection with zero connectivity	Quantized/distilled versions of #1, #3 (keyword), #6	On-device , ≤15 MB total	N/A — the alternative is nothing

Phase 0 — Research · S0 W1 · 4 days

Goal	Establish what data actually exists, what can be acquired, and which models are realistic in 22 weeks with one person.
Deliverables	Data availability audit, dataset acquisition plan, model feasibility assessment per system, 2 spike reports (crash detection from public sensor datasets; ASR quality on Indian-accented speech), labelling strategy and cost.
Expected output	An honest classification of each of the 9 systems: <i>trainable now</i> / <i>trainable with acquired data</i> / <i>rules-only in v1, ML in v2</i> .
Folder structure	<code>docs/research/ai/</code> , <code>ml/notebooks/00-feasibility/</code>
Database tables	None — but produce the feature requirements list that D1 designs the feature store around in P3.
APIs	None.
UI screens	None — but with D2, decide where AI is visible vs invisible.
Components	None.
AI models	None trained. Evaluate pre-trained options: Whisper variants, IndicWhisper, IndicBERT/MuRIL, MobileViT, EfficientNetV2, open DTC datasets, Kaggle vehicle damage datasets.
Libraries	Evaluation: PyTorch, scikit-learn, LightGBM, HuggingFace, ONNX Runtime, TFLite, MLflow, Feast.
Risks	The dominant risk of this entire vertical: no proprietary training data at project start. Mitigation: (a) every model has a rules-based v1 that ships regardless; (b) instrument for data collection from P5 onward so real data accumulates from the first booking; (c) use public/synthetic data plus transfer learning for v1 models; (d) be explicit in the pitch about which models are trained on real vs. public data — never claim otherwise.
Security	Establish the data governance position now: what can be used for training, under what consent, with what retention. This is a DPDP obligation, not a nice-to-have.
Testing	Define evaluation protocol per system: metric, held-out strategy, and — critically — the baseline each must beat.

Documentation	Data audit, feasibility assessment with the <i>trainable now / later / rules-only</i> classification, labelling plan, evaluation protocol.
Git branches	<code>docs/p00-d3-ai-feasibility</code> , <code>spike/p00-d3-crash-detection</code> , <code>spike/p00-d3-indic-asr</code>
Time	4 ideal days
Demo	Present the honest classification. Which of the 9 are real in v1, which are rules wearing an AI label, and what data would change that. This candour is what makes the rest credible.

Phase 1 — Planning · S0 W2 · 3 days

Goal	An AI backlog sequenced by value-per-effort, with the data pipeline that makes v2 possible built into v1.
Deliverables	AI epic breakdown (~35 stories), estimates, model development lifecycle definition, MLOps toolchain decision, data collection instrumentation plan.
Folder structure	<code>ml/ ├── data/{raw,processed,external,synthetic}/ ├── notebooks/ ├── src/{features,models,training,evaluation,serving}/ ├── configs/ ├── tests/ └── models/ (registry pointers, not weights)</code>
Database tables	Feature store schema handed to D1: <code>ml.feature_vehicle_daily</code> , <code>ml.feature_user_behaviour</code> , <code>ml.feature_mechanic_daily</code> , <code>ml.model_predictions</code> , <code>ml.training_snapshots</code> , <code>ml.labels</code> .
APIs	AI Gateway contract agreed with D1 (envelope, confidence, model version, fallback flag, timeout budget per capability).
AI models	None.
Libraries	Locked: PyTorch 2.x, scikit-learn, LightGBM, HuggingFace Transformers, ONNX Runtime, TFLite, MLflow, DVC, FastAPI, Feast (or a lightweight custom feature store — decide in P2).
Risks	MLOps overhead consuming the time meant for modelling → deliberately minimal toolchain: MLflow for tracking/registry, DVC for data versioning, plain Docker for serving. No Kubeflow, no Airflow, no feature-store platform. One person cannot operate that. Record as ADR-020.
Security	Data collection instrumentation designed with consent gating from the start — every training-eligible event carries the consent state at collection time.
Testing	Define the model test suite shape: data validation, training reproducibility, eval regression, inference contract.
Documentation	ML lifecycle doc, ADR-020 (minimal MLOps), data collection plan, model card template.
Git branches	<code>chore/p01-d3-ml-scaffold</code> , <code>docs/p01-d3-ml-lifecycle</code>
Time	3 ideal days
Demo	Run an end-to-end toy pipeline: data → train → MLflow registry → ONNX export → FastAPI serve → prediction. The plumbing works before any real model exists.

Phase 2 — System Architecture · S1 W3 · 3 days (support role)

Goal	Define the AI Gateway so models can be swapped, versioned, and rolled back without touching product code.
Deliverables	AI Gateway design, model serving architecture, A/B and shadow-mode design, fallback registry contract with D1.
Folder structure	<code>apps/ai-gateway/src/{routing, capabilities, models, guardrails, telemetry}/</code>
APIs	<code>POST /ai/v1/diagnose · /ai/v1/diagnose-image · /ai/v1/voice/transcribe · /ai/v1/voice/intent · /ai/v1/predict-maintenance · /ai/v1/match · /ai/v1/fraud-score · /ai/v1/forecast</code> — all sharing one envelope.
Components	Capability router (capability name → model version, resolved at request time from the registry, so a rollback is a config change not a deploy). Shadow mode (new model runs alongside the old, results logged and compared, never served). Guardrails layer: PII scrubbing on input, schema validation on output, confidence thresholding, timeout enforcement.
AI models	None yet.
Libraries	<code>fastapi</code> , <code>pydantic</code> , <code>onnxruntime</code> , <code>mlflow</code> , <code>prometheus-client</code> .
Risks	Model rollback being slow during an incident → the registry-driven router makes rollback a single config write, provable in the P15 drill.
Security	PII scrubbing before any model sees input (names, phone numbers, plate numbers redacted from free text). Model outputs validated against a schema — a model must never be able to inject arbitrary content downstream. Prompt-injection defence on any LLM path: user text is data, never instructions, and the LLM's role is limited to explaining a structured result it did not choose.
Testing	Gateway contract tests; guardrail tests (malformed output, PII leakage, timeout, injection attempt).
Documentation	AI Gateway architecture, ADR-021 (registry-driven routing), guardrail specification.
Git branches	<code>docs/p02-d3-ai-gateway-design</code>
Time	3 ideal days
Demo	Change a model version in the registry; the next request uses the new model with no deploy. Then roll it back in 5 seconds.

Phase 3 — Database Design · S1 W4 · 2 days (support role)

Goal	Make sure the OLTP schema will actually yield trainable data.
Deliverables	Feature store schema, label capture design, training snapshot strategy.
Database tables	<code>ml.feature_vehicle_daily</code> , <code>ml.feature_user_behaviour</code> , <code>ml.feature_mechanic_daily</code> , <code>ml.model_predictions</code> , <code>ml.training_snapshots</code> , <code>ml.labels</code> , <code>ml.feedback</code>
Components	Label capture is the highest-leverage design decision in this phase. Every booking outcome, every mechanic's actual diagnosis vs. our predicted one, every user "that wasn't right" — these are labels. If they aren't captured from day one, there is no v2 model. Design the capture now, even though it won't be used for months.
Risks	Training/serving skew — features computed differently in training and production → single feature definition module (<code>ml/src/features/</code>) imported by both the batch job and the serving path. Non-negotiable.

Security	Feature store contains no direct identifiers; entities are referenced by salted hash. Consent state stored alongside each row so training sets can be filtered by lawful basis.
Testing	Feature parity test: the same input through the batch path and the serving path must produce byte-identical features.
Documentation	Feature dictionary, label taxonomy, training snapshot policy.
Git branches	<code>feat/p03-d3-feature-store-schema</code> , <code>docs/p03-d3-label-taxonomy</code>
Time	2 ideal days
Demo	Show the feature parity test failing when someone changes a feature in only one place.

Phase 4 — Authentication · S2 W5 · 1 day (support role)

Goal	Ensure auth emits the signals fraud detection will need.
Deliverables	Auth event requirements handed to D1; device fingerprinting design.
Components	Signals: <code>auth.otp_failed</code> , <code>auth.device_new</code> , <code>auth.refresh_reuse</code> , <code>auth.velocity_anomaly</code> , <code>auth.msisdn_changed</code> , device fingerprint hash.
Security	Device fingerprinting must be privacy-respecting: a salted hash of stable, non-identifying attributes. No advertising ID, no cross-app tracking. Documented in the privacy policy.
Documentation	Fraud signal catalogue.
Git branches	<code>docs/p04-d3-fraud-signals</code>
Time	1 ideal day
Demo	N/A — reviewed in D1's auth demo.

Phase 5 — Backend APIs · S2–S3 · 4 days (support role)

Goal	Build the AI Gateway shell with rules-based capabilities so D1 and D2 can integrate immediately, months before the models exist.
Deliverables	AI Gateway service running, all 8 capability endpoints live returning rules-based results , prediction logging, shadow-mode infrastructure.
Expected output	Every AI endpoint the product needs exists and returns a correct, useful answer on day one — via rules. Models replace the rules behind the same contract, one at a time, with no product change.
Folder structure	<code>apps/ai-gateway/src/capabilities/{diagnose,image,voice,maintenance,match,fraud,forecast}/</code> each with <code>rules.py</code> and (later) <code>model.py</code> .
APIs	All 8 capabilities live.

Components	Rules implementations: DTC lookup + symptom keyword matching for diagnosis; distance+rating+availability scoring for matching; threshold rules for fraud; manufacturer intervals for maintenance. These are the permanent fallbacks, not throwaway stubs.
AI models	None yet — but this phase is what makes the later phases low-risk.
Libraries	<code>fastapi</code> , <code>uvicorn</code> , <code>pydantic</code> , <code>rapidfuzz</code> (symptom matching), <code>pandas</code> .
Risks	Rules being treated as temporary and left untested → they are the production fallback for the life of the product; they get the same test coverage as the models.
Security	Guardrails active from day one, before any model exists.
Testing	Capability contract tests; rules correctness tests reviewed by 2 domain-expert mechanics from P0.
Documentation	Rules documentation per capability — this doubles as the specification each model must beat.
Git branches	<code>feat/p05-d3-ai-gateway</code> , <code>feat/p05-d3-rules-capabilities</code> , <code>feat/p05-d3-prediction-logging</code>
Time	4 ideal days
Demo	Every AI feature in the app works end-to-end. None of it is a model yet. Nobody can tell from the product — and that's the point: the models are now a pure upgrade, not a dependency.

Phase 6 — Frontend · S3–S4 · 1 day (support role)

Goal	Make sure AI is presented honestly.
Deliverables	Confidence band definitions and copy, AI presentation guidance for D2, feedback capture schema.
Components	Confidence bands: ≥ 0.85 "high", $0.6-0.85$ "medium", < 0.6 "low → defer to mechanic". Each maps to specific plain-language copy in 8 languages, not a percentage.
Risks	Users over-trusting output → the copy for the low band explicitly says a mechanic will confirm, and the conservative rules verdict is always shown alongside on safety questions.
Documentation	Confidence band spec, AI copy guidelines.
Git branches	<code>docs/p06-d3-confidence-bands</code>
Time	1 ideal day
Demo	Reviewed in D2's demo.

Phase 7 — AI Development · S4 W9 – S5 W12 · 18 days · You own this phase

This is the core of your work. Broken into per-model sub-blocks.

7.1 Vehicle Diagnosis AI · 4 days

Goal	Beat the rules baseline (top-3 accuracy 0.62) on symptom → cause.
-------------	---

Inputs	Symptom text (8 languages), vehicle make/model/year/fuel, DTC codes if present, odometer, service history, recent telemetry.
Outputs	Ranked causes with probability, severity (1–5), drivability verdict, predicted parts, natural-language explanation.
Training data	Curated fault knowledge base (~4k fault patterns, built from service manuals, mechanic interviews from P0, and public DTC databases), plus synthetic symptom paraphrases generated across 8 languages and validated by native speakers.
Algorithm	Hybrid retrieval + classification: BM25 + dense retrieval (MuRIL embeddings) over the KB → LightGBM re-ranker on retrieved candidates + structured features → an LLM writes the explanation from the selected structured result only . The LLM never chooses the diagnosis.
Deployment	Server (retriever + ranker + LLM explanation); distilled classifier-only variant on-device.
Metrics	Top-1 acc, top-3 acc, severity MAE, false-"safe-to-drive" rate (must be 0) , latency p95.
Risks	Hallucinated explanations → the LLM is constrained to explain a structured result it did not pick, with the cause, severity, and parts injected as fixed facts; output is validated against those facts before it is returned.
Testing	Held-out set of 400 real cases labelled by 3 mechanics with inter-annotator agreement reported. Adversarial test: 50 prompt-injection attempts in the symptom field.
Branches	<code>model/p07-d3-diagnosis-retriever</code> , <code>model/p07-d3-diagnosis-ranker</code> , <code>model/p07-d3-diagnosis-explainer</code>

7.2 Image Diagnosis AI · 3 days

Goal	Top-3 accuracy ≥ 0.85 on damage/part identification.
Inputs	1–5 photos + vehicle context. Outputs
Training data	Public vehicle damage datasets + ~3k images collected and labelled during P0–P6 dogfooding + heavy augmentation (rain, night, motion blur, low light — the actual conditions).
Algorithm	MobileViT-S or EfficientNetV2-S backbone, multi-label part head + ordinal severity head, transfer learning from ImageNet then domain fine-tuning.
Deployment	Server FP16; INT8 quantized on-device variant (~6 MB).
Metrics	Top-1/top-3 accuracy, severity ordinal MAE, per-class recall (watch for the rare-but-serious classes), calibration (ECE).
Risks	Bias toward well-photographed cars in daylight → the augmentation strategy and a stratified eval set covering night, rain, and low-end camera quality. Report metrics per stratum, not just overall — an aggregate number hides the failure that matters.
Testing	Stratified eval by lighting, weather, camera quality, vehicle class. Calibration plot required in the model card.
Branches	<code>model/p07-d3-image-diagnosis</code> , <code>model/p07-d3-image-quantize</code>

7.3 Voice Assistant · 3 days

Goal	Intent accuracy ≥ 0.85 across 8 languages including code-switched speech.
Inputs	Speech audio. Outputs
Training data	IndicWhisper / Whisper-small fine-tuned on public Indian-accented corpora (Shrutilipi, IndicSUPERB) + ~500 in-domain utterances recorded across the team and pilot users. Intent classifier trained on templated + paraphrased in-domain utterances.

Algorithm	ASR (Whisper-small, fine-tuned) → intent classifier (MuRIL + linear head) → slot filler (token classification) → TTS (Indic TTS). On-device: keyword spotting for "help"/"SOS" in 8 languages only (~2 MB).
Deployment	Server for full pipeline; on-device keyword spotting always-on for emergency.
Metrics	WER per language, intent accuracy, slot F1, end-to-end latency, keyword-spotting FPR (critical — a false "SOS" wake is bad).
Risks	Code-switching (Hinglish) degrading both ASR and intent → include code-switched examples in training and evaluate on a dedicated code-switched set; report it separately.
Testing	Per-language WER; noisy-environment tests (highway noise at 70 dB, which is the actual use case); keyword FPR over 100 hours of ambient audio.
Branches	<code>model/p07-d3-asr-finetune</code> , <code>model/p07-d3-intent-nlu</code> , <code>model/p07-d3-keyword-spotting</code>

7.4 Predictive Maintenance · 3 days

Goal	recall@30-days $\geq$ 0.60 on component failure — roughly double the fixed-interval baseline.
Inputs	Telemetry time series (engine temp, RPM patterns, battery voltage, fuel trim, EV cell data), service history, odometer, vehicle model, usage pattern (city/highway/load).
Outputs	Per-component failure probability at 30/90 days, recommended service date, reasoning.
Training data	Synthetic + public degradation datasets (NASA C-MAPSS style adapted), plus real telemetry accumulating from P5. Honest position: v1 is largely synthetic-trained; this is stated in the model card and the pitch.
Algorithm	Survival analysis (Cox PH baseline, DeepSurv if data supports it) per component + LightGBM classifier on engineered rolling features.
Deployment	Server, nightly batch, results cached.
Metrics	Concordance index, recall@30d, precision@30d, calibration, lead time distribution.
Risks	Alarm fatigue from false positives → precision floor enforced; a prediction is surfaced only above a threshold tuned for precision, not recall, because a user who ignores three false warnings will ignore the real one.
Testing	Time-based split (never random — that leaks the future). Per-vehicle-class evaluation.
Branches	<code>model/p07-d3-predictive-maintenance</code>

7.5 Smart Mechanic Matching · 2 days

Goal	acceptance@1 $\geq$ 0.65 vs the 0.41 distance+rating baseline.
Inputs	Request (service type, vehicle, location, urgency, time), mechanic features (skills, ratings, inventory, historical acceptance by context, current load, distance, predicted travel time), plus contextual features (traffic, weather, time of day).
Outputs	Ranked mechanics with acceptance probability and predicted ETA.
Training data	Historical dispatch outcomes from P5 onward. Cold start: rules-based, transitioning to ML once ~5k offers are accumulated.
Algorithm	LambdaMART (LightGBM ranker) for ranking + separate gradient-boosted ETA regressor.
Deployment	Server, hard 300 ms budget, fallback to rules on timeout.

Metrics	acceptance@1, NDCG@5, ETA MAE, fairness: Gini coefficient of job distribution across mechanics.
Risks	The model concentrating all jobs on a few high-performing mechanics, starving new entrants and destroying supply → explicit exploration term (epsilon-greedy) plus a fairness constraint monitored as a first-class metric. A matching model that optimizes only for acceptance will kill the marketplace.
Testing	Offline replay on historical data; online A/B via shadow mode before serving; fairness metric tracked per week.
Branches	<code>model/p07-d3-mechanic-ranker</code> , <code>model/p07-d3-eta-regressor</code>

7.6 Crash Detection · 3 days

Goal	recall ≥ 0.95 , FPR ≤ 1 per 1,000 driving hours, on-device, <120 ms. The highest-stakes model in the project.
Inputs	100 Hz accelerometer, gyroscope, GPS speed, barometer — 5-second sliding windows.
Outputs	Crash probability + severity estimate.
Training data	Public crash sensor datasets, controlled drop/impact tests, simulated crash signatures, plus a large corpus of <i>negative</i> data — normal driving on Indian roads, which contains potholes, speed breakers, and sudden braking that all look somewhat like crashes. The negative data is harder to get right than the positive data and matters more.
Algorithm	1D CNN feature extractor + GRU temporal model, INT8 quantized, ~1.5 MB.
Deployment	On-device only. Never server-side — it must work with no connectivity, which is exactly when a highway crash happens.
Metrics	Recall, FPR per driving hour, severity MAE, inference latency, battery cost per hour.
Risks	This is R3, a critical project risk. A false positive dispatches real emergency services. Mitigations, layered: (a) the model raises a <i>signal</i> , never an incident; (b) 30 s user-cancellable countdown; (c) call-back verification before any 112 handoff; (d) the model cannot auto-dispatch — this is enforced in D1's code, not just policy; (e) speed-breaker and pothole signatures explicitly in the negative training set; (f) FPR measured over real driving hours during dogfooding, not just on a test set.
Security	Sensor data stays on-device unless a crash is confirmed. No continuous sensor upload — that would be both a privacy violation and a battery disaster.
Testing	Controlled tests: 50 speed breakers, 30 potholes, 20 hard-braking events, 10 phone drops, 5 phone-thrown-on-seat events — all must produce zero crash signals. Recall measured on labelled crash traces. 200 hours of real driving during dogfooding with FPR measured.
Branches	<code>model/p07-d3-crash-detection</code> , <code>model/p07-d3-crash-quantize</code>

7.7 Fraud Detection · 2 days

Goal	Precision ≥ 0.70 at recall 0.50, versus the 0.34-precision rules baseline.
Inputs	Booking graph (user↔mechanic↔device↔payment relationships), payment patterns, GPS trace plausibility, review patterns, auth signals from P4, timing patterns.
Outputs	Risk score 0–100 + reason codes.
Training data	Confirmed fraud cases (few, initially) + synthetic fraud injection + unsupervised anomaly detection to bootstrap labels for human review.
Algorithm	Graph feature extraction (community detection for collusion rings) + isolation forest for unsupervised anomaly + LightGBM supervised model as confirmed cases accumulate.

Deployment	Server, async — never blocking a booking.
Metrics	Precision@k, recall on confirmed cases, false-accusation rate (the metric that matters most).
Risks	Falsely flagging a legitimate mechanic, destroying their livelihood → the model never auto-suspends. It queues for human review with reason codes, and the mechanic is told the outcome. Reason codes are mandatory: an unexplainable fraud score is unusable.
Security	Fraud scores are internal only, never exposed to users or mechanics. Access to the fraud queue is role-restricted and audited.
Testing	Synthetic fraud injection (collusion rings, GPS spoofing, review farming); false-positive review with the ops team.
Branches	<code>model/p07-d3-fraud-graph-features</code> , <code>model/p07-d3-fraud-model</code>

7.8 Model serving & registry · 2 days

Wire all trained models into the AI Gateway behind the existing rules contracts, with shadow mode, A/B, and one-config rollback. Prometheus metrics per model: latency, throughput, confidence distribution, fallback rate, and drift indicators.

Phase 7 summary

Deliverables	7 trained models, model registry populated, 7 model cards, eval harness, shadow-mode results, on-device bundle started.
Libraries	<code>torch</code> , <code>transformers</code> , <code>lightgbm</code> , <code>scikit-learn</code> , <code>lifelines</code> , <code>onnxruntime</code> , <code>optimum</code> , <code>mlflow</code> , <code>dvc</code> , <code>librosa</code> , <code>alumentations</code> , <code>networkx</code> .
Security	No PII in any training set without a documented lawful basis. Model weights are artefacts in the registry, not in git. Every model card includes a bias evaluation across vehicle class, language, and region.
Testing	Every model must beat its documented baseline on a held-out set. Every model has a calibration report. Every model has an adversarial/robustness test appropriate to its input type.
Documentation	7 model cards, evaluation report, training reproducibility guide (<code>dvc repro</code> reproduces any model from its commit).
Time	18 ideal days
Demo	For each model: the baseline, the model, the delta, the failure modes, and a live inference. Then unplug the model and show the rules fallback keeping the product working.

Phase 8 — Offline Engine (on-device AI) · S5 W12 · 4 days

Goal	Fit useful intelligence into 15 MB and 120 ms on a 2 GB phone.
Deliverables	On-device model bundle (crash detection, diagnosis classifier, keyword spotting), TFLite/ONNX Runtime Mobile integration, on-device inference benchmarks, model update channel.
Folder structure	<code>ml/src/serving/ondevice/</code> , <code>packages/ml-ondevice/</code> (RN native module)
Components	Bundle budget: crash detection 1.5 MB · diagnosis classifier 8 MB · keyword spotting 2 MB · DTC lookup table 3 MB = 14.5 MB . Models delivered out-of-band (downloaded on first run, not in the APK) so the APK stays small and models can update independently of app releases.

AI models	Quantized/distilled variants of #1, #3 (keyword only), #6.
Libraries	<code>onnxruntime-mobile</code> , <code>tensorflow-lite</code> , <code>react-native-fast-tflite</code> , <code>optimum</code> (quantization), <code>torch.ao.quantization</code> .
Risks	(a) Quantization degrading accuracy below usefulness → measure post-quantization accuracy explicitly; if the drop exceeds 3 points, distil instead of quantizing further. (b) Model download failing on poor connectivity → the app is fully functional without on-device models (falls back to server, then to rules); the models are an enhancement, never a requirement.
Security	Models are signed and verified before loading — an unverified model file is a code execution vector. Model files stored in app-private storage.
Testing	Benchmark on 5 real low-end devices: latency, memory peak, battery cost, thermal behaviour under sustained inference. Accuracy comparison quantized vs full precision.
Documentation	On-device deployment guide, quantization report (accuracy before/after per model), device benchmark table.
Git branches	<code>feat/p08-d3-ondevice-bundle</code> , <code>feat/p08-d3-tflite-native-module</code> , <code>feat/p08-d3-model-update-channel</code>
Time	4 ideal days
Demo	Airplane mode on a ₹8,000 phone. Describe a symptom, get a diagnosis in under 200 ms. Simulate a crash, get detection on-device. Nothing touches the network.

Phase 9 — Maps · S6 W13 · 1 day (support role)

Goal	Improve ETA beyond what the routing engine gives.
Deliverables	ETA correction model — Valhalla's ETA adjusted by learned residuals from historical actuals.
Components	Features: route distance, time of day, day of week, weather, historical corridor speed, mechanic's vehicle type, monsoon flag. Target: actual – predicted travel time.
Metrics	ETA MAE improvement vs raw routing engine; target ≥25% reduction.
Risks	Insufficient historical data early → ship the raw routing ETA with an honest range ("15–25 min") rather than a false precision ("18 min") until the residual model has data.
Testing	Time-based split; per-corridor evaluation.
Branches	<code>model/p09-d3-eta-correction</code>
Time	1 ideal day

Phase 10 — Vehicle Diagnostics · S6 W14 · 5 days · You own this phase (with D2)

Goal	Extend diagnosis to the vehicles that don't have OBD — which in India is most of them.
Deliverables	Two-wheeler / auto / tractor rule packs, EV diagnostic module, sound-based diagnosis spike, DTC → plain language in 8 languages, integrated diagnosis pipeline.

Components	Per-vehicle-class rule packs built with the mechanics interviewed in P0 — this is knowledge engineering, not machine learning, and for two-wheelers it's the right tool. EV module: SoC anomaly, cell imbalance, thermal events, charging faults, range estimation. Sound diagnosis: timeboxed 1-day spike (mel-spectrogram + small CNN on engine/brake noise) — ships in v1.5 only if the spike succeeds.
AI models	Diagnosis model extended with vehicle-class conditioning; sound classifier if the spike passes.
Metrics	Per-vehicle-class top-3 accuracy; mechanic satisfaction score — 5 mechanics rate 50 outputs each, which is a more honest measure than accuracy on a curated set.
Risks	Two-wheeler diagnosis being weak because there's no telemetry → lean entirely on structured symptom questioning plus photo/sound, and be honest in the UI that it's a triage, not a diagnosis.
Security	Diagnostic photos scrubbed of EXIF location before storage or training use.
Testing	Per-class held-out sets; the mechanic rating exercise; the zero-false-"safe" assertion across all classes.
Documentation	Rule pack documentation per vehicle class, EV diagnostic spec, sound diagnosis spike report.
Git branches	<code>model/p10-d3-vehicle-class-rules</code> , <code>model/p10-d3-ev-diagnostics</code> , <code>spike/p10-d3-sound-diagnosis</code>
Time	5 ideal days
Demo	Diagnose a scooter that won't start, using only structured questions and a photo. Then have a real mechanic grade the answer live.

Phase 11 — Emergency Engine · S7 W15 · 3 days (support role)

Goal	Get crash detection to production quality and validate severity estimation.
Deliverables	Crash detection deployed on-device, severity estimation model, false-positive dataset from dogfooding, tuned thresholds.
Components	Threshold tuning against the dogfooding FPR data. Severity estimation feeding the escalation ladder (severity ≥ HIGH escalates faster). Two-signal corroboration logic: sensor signal + no user response + abnormal post-event motion pattern.
Risks	R3 again, now with real stakes. Final gate before production: FPR measured over ≥200 real driving hours across ≥5 devices and ≥3 road types must be ≤ 1 per 1,000 hours. If it isn't, crash detection ships disabled by default and manual SOS carries the feature. That decision is pre-committed here so it isn't a judgement call under launch pressure.
Testing	The full controlled test battery from 7.6, re-run on the production build. Dogfooding FPR report.
Documentation	Crash detection model card updated with production FPR; the ship/no-ship decision recorded.
Git branches	<code>model/p11-d3-crash-threshold-tuning</code> , <code>model/p11-d3-severity-estimation</code>
Time	3 ideal days
Demo	Present the real dogfooding FPR number. Drive over a speed breaker on camera. Nothing happens. That's the demo.

Phase 12 — Government Dashboard · S7 W16 · 3 days (support role)

Goal	Forecasting and blackspot analysis that a transport department would act on.
Deliverables	Blackspot detection model, spatiotemporal incident forecasting, corridor risk scoring, demand prediction.
Components	Blackspot detection: spatial clustering (DBSCAN on incident locations) weighted by severity, with statistical significance testing so a cluster isn't just traffic volume. Forecasting: temporal fusion transformer or a well-tuned gradient-boosted model on spatiotemporal features (historical incidents, weather, festival calendar, road geometry, traffic volume).
Metrics	Blackspot precision against known official blackspot lists (a genuinely strong validation signal — if we rediscover the blackspots the state already knows about, plus some it doesn't, that's credible). Forecast MAE vs a historical-average baseline.
Risks	Forecasts driving real infrastructure spending → outputs carry confidence intervals and an explicit methodology note; we present ranges, never point estimates, to a government user.
Security	All outputs are aggregate and k-anonymized before they reach the gov service. The model never sees, and never emits, individual identifiers.
Testing	Validation against published official blackspot data for one state. Time-based forecast backtesting.
Documentation	Blackspot methodology, forecasting model card, a limitations section written for a non-technical government reader.
Git branches	<code>model/p12-d3-blackspot-detection</code> , <code>model/p12-d3-incident-forecasting</code>
Time	3 ideal days
Demo	Run blackspot detection on a state's historical data; overlay the official blackspot list; show the overlap and the new candidates.

Phase 13 — Analytics · S8 W17 · 5 days · You own this phase

Goal	An analytics layer where every number has exactly one definition and nobody argues about which dashboard is right.
Deliverables	Event taxonomy, ClickHouse schema, 4 dashboard suites (with D2), metric dictionary, anomaly alerting, cohort/funnel analysis.
Folder structure	<code>ml/src/analytics/</code> , <code>infra/clickhouse/</code>
Database tables	ClickHouse: <code>events</code> , <code>bookings_fact</code> , <code>incidents_fact</code> , <code>mechanic_daily</code> , <code>vehicle_daily</code> , <code>funnel_steps</code> , <code>cohorts</code>
Components	Metric dictionary is the deliverable that matters most here. One definition per metric, versioned, referenced by every dashboard. Anomaly detection on business metrics (seasonal decomposition + threshold on residuals) so a 40% drop in bookings pages someone.
Metrics tracked	Demand (requests/hour by geo), supply (active mechanics), match rate, time-to-match, cancellation rate and reasons, completion rate, NPS, revenue, take rate, CAC, LTV, cohort retention, SLO attainment, incident response times, mechanic utilization and earnings distribution.

Risks	Metric definitions drifting between dashboards → the dictionary is enforced by a shared SQL macro layer; dashboards cannot define their own aggregations ad hoc.
Security	Zero PII in ClickHouse, enforced by a CI scan of the schema. Analytics access role-restricted.
Testing	Metric correctness tests (compute each metric two ways and assert agreement). Pipeline lag test. Anomaly detection tested against injected synthetic anomalies.
Documentation	Event taxonomy, metric dictionary, dashboard guide, anomaly alerting runbook.
Git branches	<code>feat/p13-d3-event-taxonomy</code> , <code>feat/p13-d3-clickhouse-schema</code> , <code>feat/p13-d3-metric-dictionary</code> , <code>feat/p13-d3-anomaly-alerting</code>
Time	5 ideal days
Demo	Inject a synthetic 40% booking drop. The anomaly alert fires within 10 minutes with the affected segment identified.

Phase 14 — Testing · S8 W18 · 4 days

Goal	Prove the models are correct, fair, robust, and safe to fail.
Deliverables	Model test suite in CI, drift detection, bias audit across all models, robustness testing, adversarial testing, model card finalization.
Components	Data validation (Great Expectations) on every training input. Eval regression tests — a model version cannot be promoted if it regresses any tracked metric. Drift detection (PSI/KL on feature distributions) alerting in production. Bias audit: performance by vehicle class, language, region, and device tier for every model; a model that works well only for car owners in metros is not shippable.
Risks	Silent model degradation in production → drift alerting plus weekly automated eval on a held-out set that refreshes with new labelled data.
Security	Adversarial testing: prompt injection on the LLM path, adversarial images on the vision model, sensor spoofing on crash detection, GPS spoofing on fraud detection.
Testing	Full suite in CI. Fairness metrics reported per model.
Documentation	All 9 model cards finalized with bias audit results; AI risk assessment.
Git branches	<code>test/p14-d3-model-test-suite</code> , <code>test/p14-d3-bias-audit</code> , <code>test/p14-d3-adversarial</code>
Time	4 ideal days
Demo	Present the bias audit. Show where each model is weakest and what we're doing about it. A team that knows its models' weaknesses is more credible than one claiming they have none.

Phase 15 — Deployment · S9 W19 · 3 days

Goal	Models deployable and rollback-able without drama.
-------------	--

Deliverables	Model serving in production (Triton or FastAPI+ONNX), GPU/CPU sizing decision, model CI/CD, monitoring, on-device model distribution.
Components	Serving on CPU with ONNX Runtime where possible (dramatically cheaper than GPU and sufficient for our latency budgets except the LLM path). Model promotion pipeline: registry → shadow → canary 5% → 100%, with automatic rollback on a metric regression.
Risks	GPU cost → measured decision: only the LLM explanation path needs GPU, and it's off the critical path with a template-based fallback. Everything else is CPU.
Security	Model artefacts signed; serving containers scanned; no model loaded from an unverified source.
Testing	Rollback drill: promote a deliberately bad model, watch the automatic rollback trigger.
Documentation	Model deployment runbook, serving architecture, cost model per 1M inferences.
Git branches	<code>feat/p15-d3-model-serving</code> , <code>feat/p15-d3-model-cicd</code>
Time	3 ideal days
Demo	Promote a bad model on purpose. Watch it get rolled back automatically in under 2 minutes.

Phase 16 — Optimization · S9 W20 · 3 days

Goal	Cheaper and faster inference without losing accuracy.
Deliverables	Inference optimization, batching, caching, distillation, cost-per-inference model.
Components	Dynamic batching on server models; result caching for deterministic inputs (same DTC + same vehicle model = same diagnosis); distillation of the LLM explanation path into a template + small model for the 80% of common cases; INT8 across all server models where accuracy permits.
Metrics	Latency p95 per capability, cost per 1M inferences, accuracy delta from each optimization (must be reported, never assumed to be zero).
Risks	Optimization silently degrading accuracy → every optimization PR includes a before/after eval on the held-out set. A PR that doesn't report it is rejected.
Documentation	Optimization report with accuracy/latency/cost tradeoffs made explicit.
Git branches	<code>perf/p16-d3-inference-optimization</code> , <code>perf/p16-d3-llm-distillation</code>
Time	3 ideal days
Demo	Cost per 1M inferences, before and after. And the accuracy table showing what it cost us.

Phase 17 — Future Roadmap · S10 · 3 days

Goal	Define the AI roadmap that the data we're now collecting makes possible.
Deliverables	AI roadmap (18 months), data strategy, model improvement plan, research agenda.

Components	The core argument: v1 models are largely bootstrapped from public and synthetic data. From launch, real labelled data accumulates from every booking (predicted vs actual diagnosis), every mechanic correction, and every user feedback tap. The roadmap is a data-flywheel plan: which model improves first, at what data volume, and what that improvement is worth. Research agenda: multimodal diagnosis (photo + sound + telemetry jointly), federated learning for on-device personalization without centralizing data, road condition mapping from aggregated accelerometer traces, V2X readiness.
Documentation	AI roadmap with data-volume thresholds per model, data strategy, research agenda, honest v1 capability statement for the pitch deck.
Git branches	<code>docs/p17-d3-ai-roadmap</code>
Time	3 ideal days
Demo	The flywheel: here is what each model does today, here is the data volume at which it gets meaningfully better, and here is when we expect to reach it.

D3 Effort Summary

Phase	Days	Phase	Days
P0 Research	4	P9 ETA model	1
P1 Planning	3	P10 Diagnostics	5
P2 AI Gateway design	3	P11 Emergency	3
P3 Feature store	2	P12 Gov forecasting	3
P4 Fraud signals	1	P13 Analytics	5
P5 Gateway + rules	4	P14 Testing	4
P6 AI UI support	1	P15 Deployment	3
P7 AI Development	18	P16 Optimization	3
P8 On-device AI	4	P17 Roadmap	3
		Total	70 ideal days

The lower total is deliberate: model work is research-shaped and the estimates are less reliable than engineering estimates. The extra slack absorbs the experiments that don't work — and some won't.